



- Parfois on est appelé à introduire des modifications dans le schéma de la base tels que :
la modification de la structure d'une table, l'ajout de nouvelles tables ou même la suppression de tables.
- Pour permettre aux procédures (fonctions) existantes de prendre en compte ces modifications, il faut les recompiler avec la commande suivante :

```
Alter { function/Procedure } <nom_p> compile ;
```

Exemple

```
Alter procedure reduire_prix compile ;  
Alter function Approvisionner compile ;
```



► Comme tout objet manipulé par oracle, les procédures et les fonctions peuvent être supprimées si le besoin est ressenti.

```
Drop { function/Procedure } <nom_p> ;
```

Exemple

```
Drop procedure reduire_prix ;
```



► Il s'agit d'un sous-programme PL/SQL **nommé et défini** dans la section **DECLARE**.

- Ce sous programme **peut avoir des paramètres**.
- Il adhère aux **mêmes règles de portée et de visibilité que les variables**.
- Il est **réutilisable** et facilite la maintenance des applications.

► Nous distinguons deux types de sous programmes non stockés : les procédures et les fonctions.

```
DECLARE -- bloc emboîtant
  procedure <nom_procédure> [(paramètre,
    [paramètre...])] {IS | AS}
    -- Déclaration locales de variables, constant, ...
  BEGIN
    -- Instructions exécutables
  [EXCEPTION
    -- Gestion des exceptions ]
  END [nom_procédure] ;

...
BEGIN
...
EXCEPTION
...
END ;
```



► Puisque les noms des sous-programmes locaux sont des identificateurs, ils **doivent être déclarés avant qu'ils soient utilisés**.

- Par exemple, si un sous programme A appelle un sous programme B, B doit être déclaré avant A.

► Dans le cas de **références mutuelles** (A appelle B et B appelle A), il faut procéder à une déclaration en avant (forward) de l'un ou même des deux sous programmes:

```
FUNCTION/PROCEDURE    <nom_fonction>    [(paramètre 1,  
[paramètre 2...]))];
```

Exercice

Les sous-programmes



Soit la BD suivante :

ARTICLE (nart, desart, pu, qtes)

CLIENT (ncl, nomcl, adr)

FACTURE (nf, datef, #ncl, mntf)

DET_FACT (#nf, #nart, qtef)

1. Créer une fonction CA_CLT_ART permettant de calculer le chiffre d'affaire d'un client donné et pour un article donné.
2. Créer une fonction CA_CLT permettant de calculer le chiffre d'affaire d'un client donné.
Le chiffre d'affaire n'est d'autre que la somme des montants factures (MNTF).
3. Créer une fonction NB_VENTE permettant de calculer le nombre de ventes d'un article donné.
4. Créer une procédure AUGMENTER permettant d'augmenter le prix unitaire d'un article donné selon les règles suivantes :
 - a) Si l'article était vendu plus que 100 fois alors l'augmentation est de 15%
 - b) Si le nombre de vente est compris entre 50 et 100 fois, alors l'augmentation est de 10%.
 - c) Sinon l'augmentation est de 5%.
5. Refaire la question N°4, mais appliquer ce traitement pour tous les articles.

Chapitre VI:

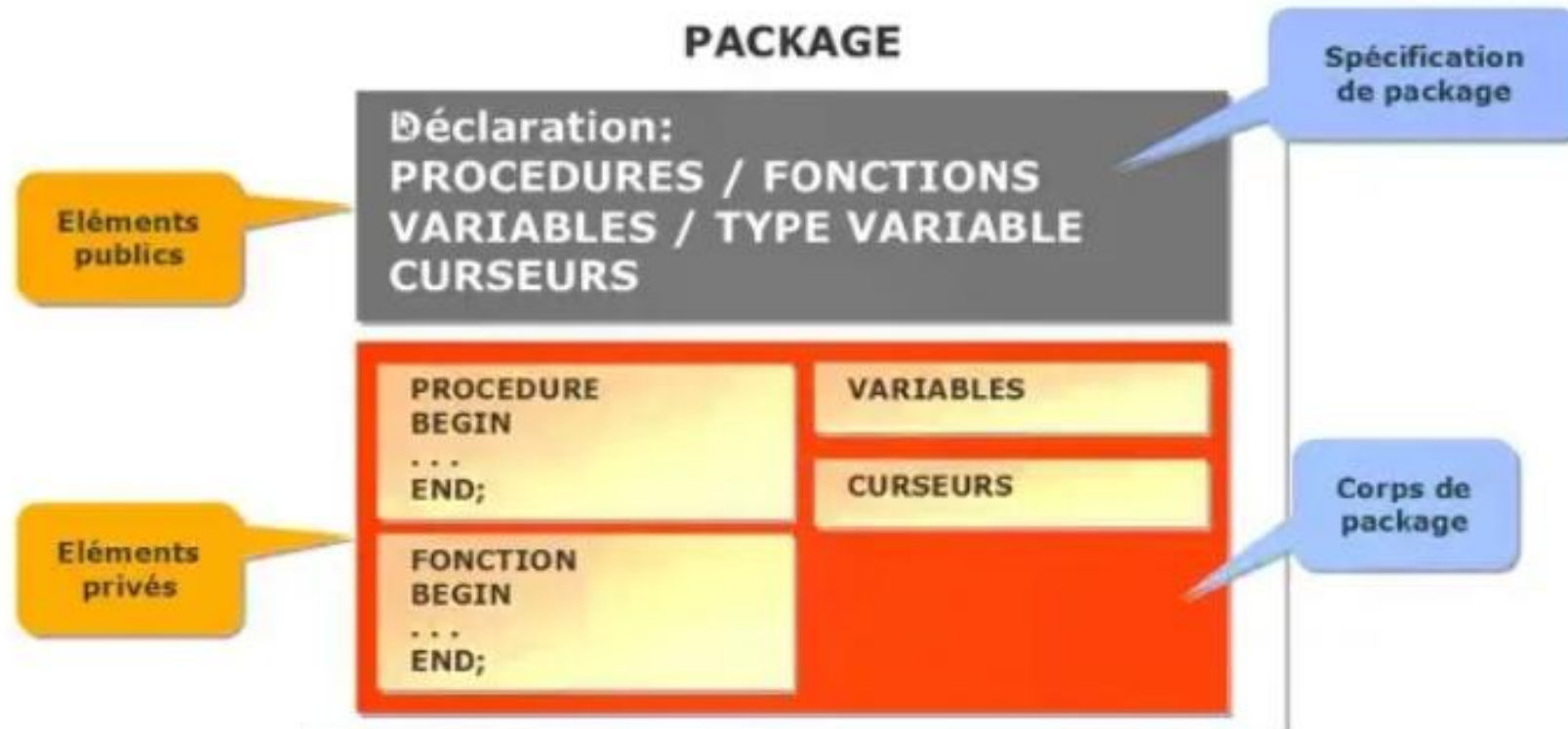
Les packages



- ▶ Un paquetage (package) est similaire à la notion de classe dans l'orienté objet.
- ▶ C'est un regroupement de définitions de procédures, fonctions, variables, constantes, curseurs et exceptions.
- ▶ Ces éléments logiciels sont regroupés parce qu'ils sont souvent utilisés ensemble (ils ont un lien logique entre eux). Ils sont stockés dans le référentiel Oracle.
- ▶ Un package est composé de deux parties créées séparément mais complémentaires:
 - Une partie de déclaration (dite aussi de spécification) ([package specification](#))
 - Une partie de traitement appelée corps du package ([package body](#)).



Structure d'un package





```
CREATE [OR REPLACE] PACKAGE <nom_package>  
{IS | AS}  
{ [prototype de procédure; ]  
  | [prototype de fonction; ]  
  | [déclaration de variable; ]  
  | [déclaration de constante; ]  
  | [déclaration de curseur; ]  
  | [déclaration d'exception; ] ...}  
END [nom_package] ;
```

La création de la partie spécification du package s'agit de la **déclaration** des objets de type **PUBLIC** du package.



Exemple :

Soit la table EMP(Numemp, Nomemp, Foncemp, salemp)

```
CREATE OR REPLACE PACKAGE Pkg_Finance
IS
-- Variables globales et publiques
GSalaire EMP.salemp%Type ;
-- Fonctions publiques
FUNCTION F_Test_Augmentation
( Pnumemp EMP.Numemp%Type , Pourcent IN NUMBER )Return
NUMBER ;
-- Procédures publiques
PROCEDURE Test_Augmentation
( PNumemp IN EMP.Numemp%Type, Pourcent IN NUMBER ) ;
End Pkg_Finance ;
```



Le corps (body) regroupe (implémente): le code public des procédures et fonctions définies dans la partie spécification le code privé des procédures et fonctions internes à la partie body ne pouvant être atteintes par les applications utilisant le package (principe de l'encapsulation)

```
CREATE [OR REPLACE] PACKAGE BODY <nom_package>
```

```
[ IS | AS ]
```

```
{ [ Déclaration de variables ; ]
```

```
| [ Déclaration de constante ; ]
```

```
| [ Implémentation de procédure ; ]
```

```
| [ Implémentation de fonction ; ]
```

```
| [ déclaration de curseur ; ]
```

```
| [ déclaration d'exception ; ] ... }
```

```
[ BEGIN ]
```

```
/* Section optionnelle contenant des commandes à exécuter une seule fois  
suite au premier appel d'un élément du package */
```

```
END [nom_package] ;
```

Ces objets sont alors "**privés**" (**PRIVATE**) et **ne peuvent en aucun cas être accédés en dehors du corps du package.**



Exemple :

```
CREATE OR REPLACE PACKAGE BODY Pkg_Finance IS  
-- Variables globales privées  
E EMP%Rowtype ;  
-- Procédure privée  
PROCEDURE Affiche_Salaires  
IS  
CURSOR C_EMP IS select * from EMP ;  
BEGIN  
OPEN C_EMP ;  
Loop  
FETCH C_EMP Into E ;  
Exit when C_EMP%NOTFOUND ;  
dbms_output.put_line( 'Employé ' || E.Nomemp || ' --> ' || Lpad( To_char(E.salemp ), 10 ) ) ;  
End loop ;  
CLOSE C_EMP ;  
END Affiche_Salaires ;
```



Exemple :

```
-- Fonctions publiques
FUNCTION F_Test_Augmentation
( PNumemp IN EMP.Numemp%Type , Pourcent IN NUMBER ) Return NUMBER IS
Salaire EMP.salemp %Type ;
BEGIN
Select salemp Into Salaire From EMP Where Numemp = PNumemp ;
-- augmentation virtuelle de l'employé
Salaire := Salaire * Pourcent ;
-- Affectation de la variable globale publique
GSalaire := Salaire ;
Return(Salaire ) ; -- retour de la valeur
END F_Test_Augmentation;
```



Exemple :

```
-- Procédures publiques
PROCEDURE Test_Augmentation
(PNumemp IN EMP.Numemp%Type , Pourcent IN NUMBER ) IS
Salaire EMP.salemp%Type ;
BEGIN
Select salemp Into Salaire From EMP Where Numemp = PNumemp ;
-- augmentation virtuelle de l'employé
Salaire := Salaire * Pourcent ;
GSalaire := Salaire ;
-- appel procédure privée
Affiche_Salaires ; -- Qu'est ce que vous remarquez ?
END Test_Augmentation;
END Pkg_Finance;
```



Remarques:

ℒ La spécification du paquetage est créée avec :

- ✓ une variable **globale** et **publique** : **GSalaire**
- ✓ une procédure **publique** : **PROCEDURE Test_Augmentation**
- ✓ une fonction **publique** : **FUNCTION F_Test_Augmentation**

→ Ces éléments sont **visibles** depuis **l'extérieur** (le programme appelant)

ℒ Le corps du paquetage est créé avec :

- ✓ une **procédure privée** : **PROCEDURE Affiche_Salaires** qui **n'est visible que dans le corps du paquetage**.
- ✓ une **variable globale** au corps du paquetage : **E** utilisée par la procédure privée.



- Seuls les objets et les sous programmes publics peuvent être référencés depuis l'extérieur

```
nomPaquetage.nom_objet
```

```
nomPaquetage.nom_sousProgramme(...)
```

- L'accès à un objet d'un paquetage est réalisé avec la syntaxe suivante:

```
nom_paquetage.nom_objet[(liste paramètres)] ;
```

- L'exécution d'une procédure d'un paquetage est réalisé avec la syntaxe s'effectue par la commande:

```
EXECUTE nom_paquetage.nom_procedure [(liste paramètres effectifs)] ;
```




-- Appel de la fonction F_Test_Augmentation du paquetage :

Declare

Salaire emp.salemp%Type ;

Begin

Select salemp Into Salaire From EMP Where Numemp = 7369 ;

dbms_output.put_line('Salaire de 7369 avant augmentation' || To_char(Salaire)) ;

dbms_output.put_line('Salaire de 7369 après augmentation' ||

To_char(**Pkg_Finance.F_Test_Augmentation(7369, 1.1)**)) ;

End ;



-- Appel de la procédure Test_Augmentation du paquetage

Declare

Pourcent NUMBER := 1.1 ;

Begin

Pkg_Finance.Test_Augmentation(7369, Pourcent) ;

dbms_output.put_line('Employé 7369 après augmentation : ' ||

To_char(Pkg_Finance.Gsalaire)) ;

End ;

-- Interrogation de la variable globale publique : Gsalaire

Begin

dbms_output.put_line(' Valeur salaire du package : ' || To_char(**Pkg_Finance.GSalaire**)) ;

End ;